

AXI4_LITE +SPI/I2C

CONTENTS

01. OVERVIEW

02. AXI4-LITE

03. SPI

04. I2C

05. CONCLUSION

Overview

01. Introduction

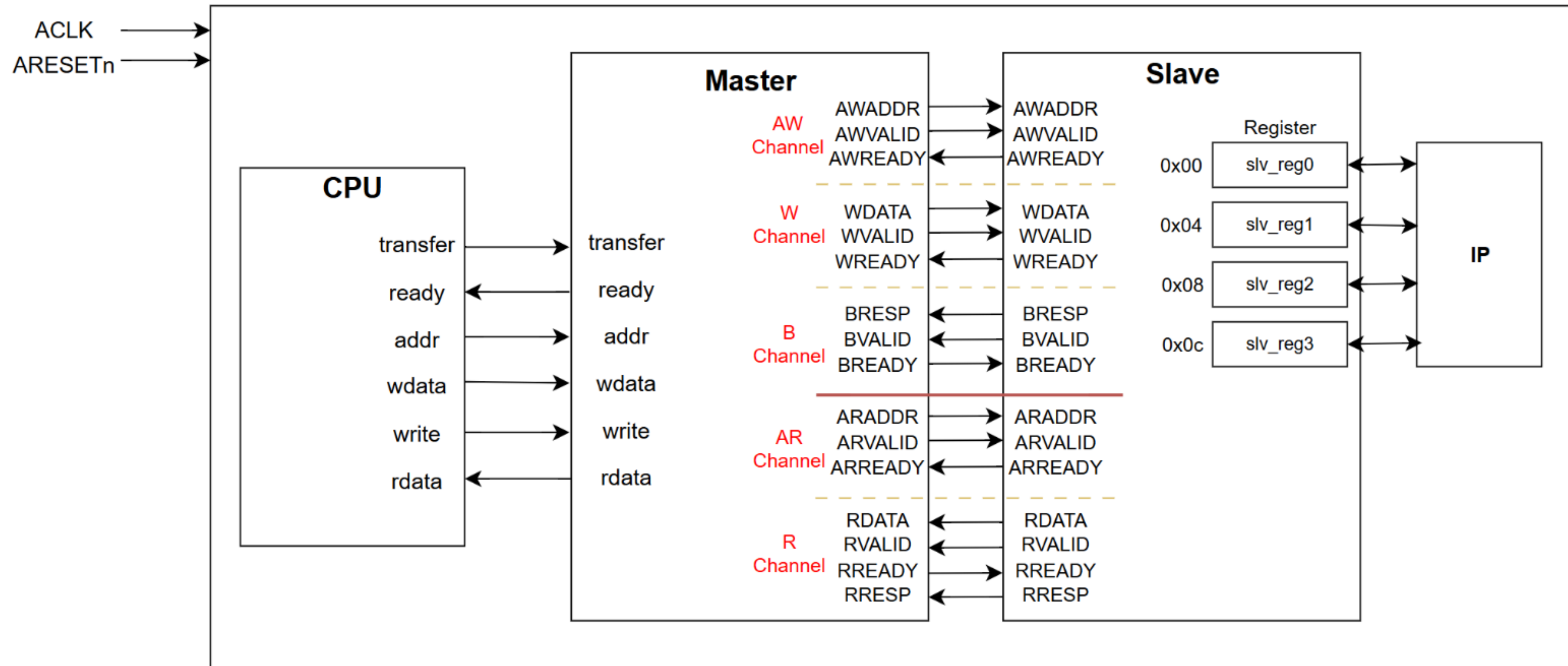
- 표준 버스 인터페이스의 이해
임베디드 시스템 내 CPU와 주변장치 간 연결에 필수적인 AXI4-Lite 프로토콜의 매커니즘 습득
 - 다양한 직렬 통신 규격 활용
하드웨어 시스템 설계에서 범용적으로 사용되는 SPI 및 I2C 통신 프로토콜의 설계 방식 및 특성 이해
-

02. Objective

- RTL Design: AXI4-Lite 기반의 SPI/I2C Master IP 설계 및 SystemVerilog를 이용한 Independent Slave 모듈 구현
 - Bus Integration: 설계된 통신 모듈을 AXI4-Lite 슬레이브 인터페이스에 매핑하여 메모리 맵 기반의 제어 환경 구축
 - Hardware Verification: 두 개의 FPGA 보드를 활용한 Board-to-Board 통신 실증 및 데이터 전송 무결성 검증
-

AXI4_LITE

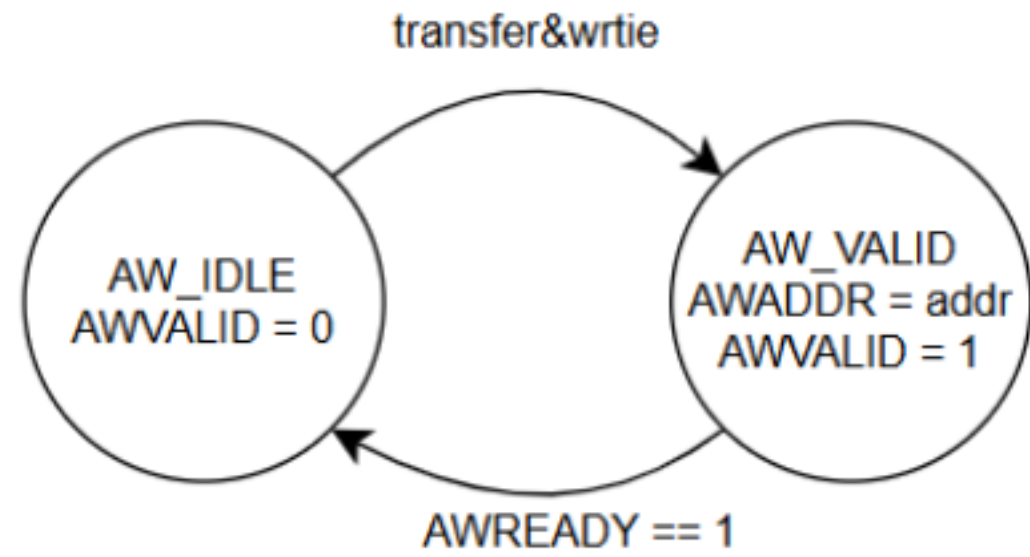
AXI Block Diagram



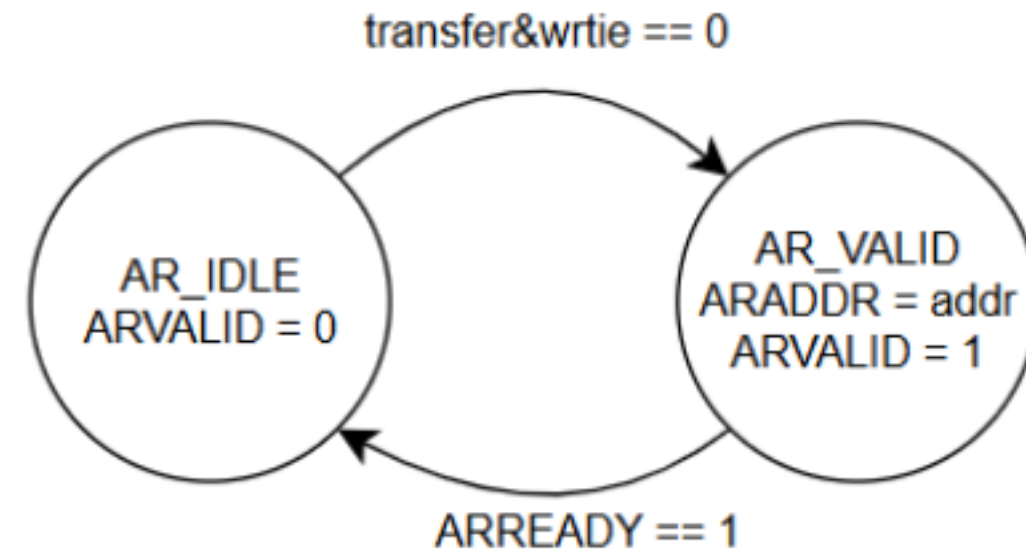
- 5-Channel Architecture: 읽기(Read)와 쓰기(Write) 채널, 그리고 주소(Address)와 데이터(Data) 채널의 분리
- Register Mapping: AXI Slave가 받은 데이터가 내부의 실제 레지스터(slv_reg0 ~ slv_reg3)에 어떻게 연결되는지 가시화

AXI4-Lite Transaction FSM

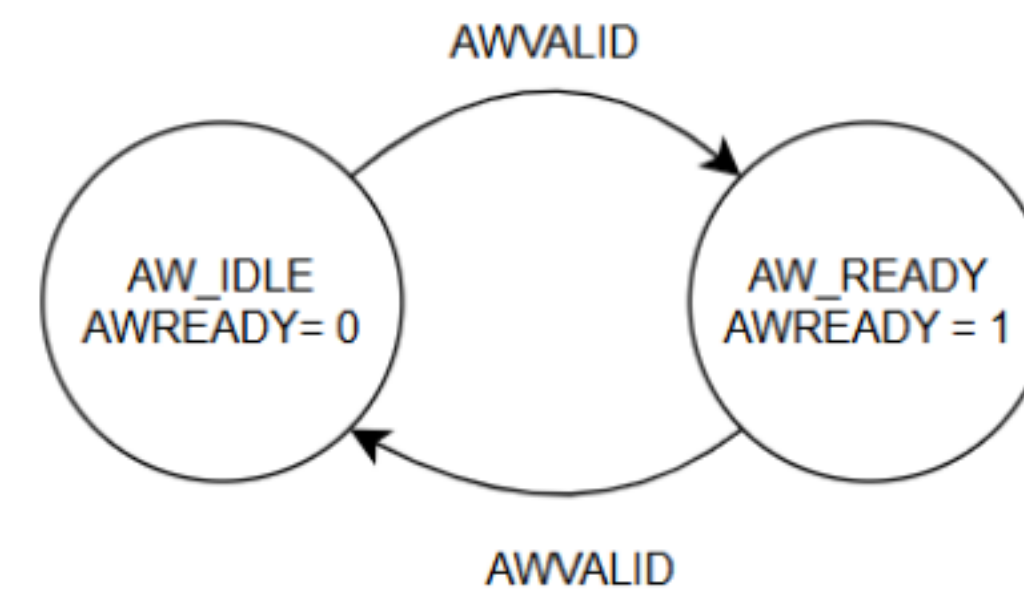
<Master> Write Transaction



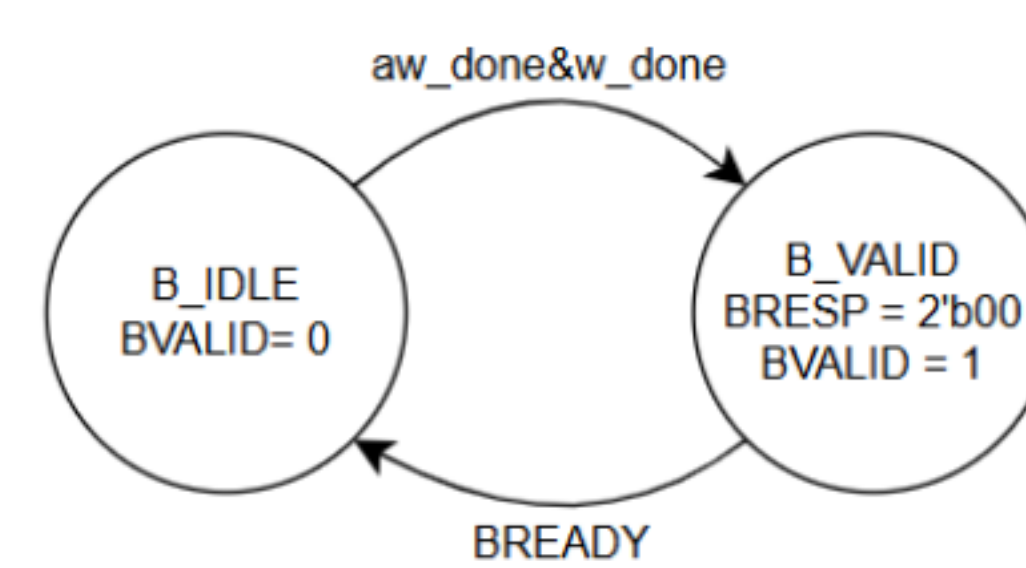
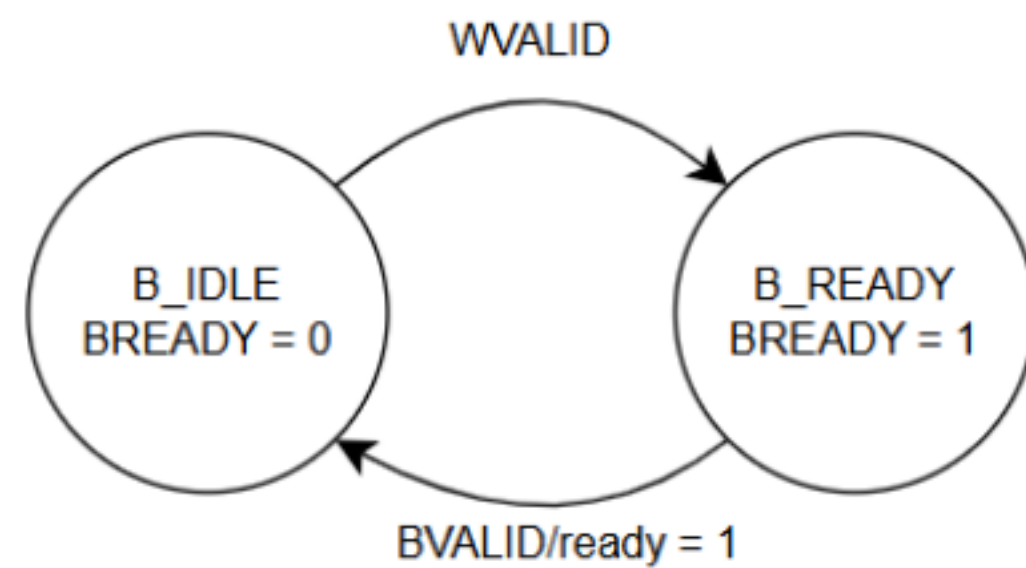
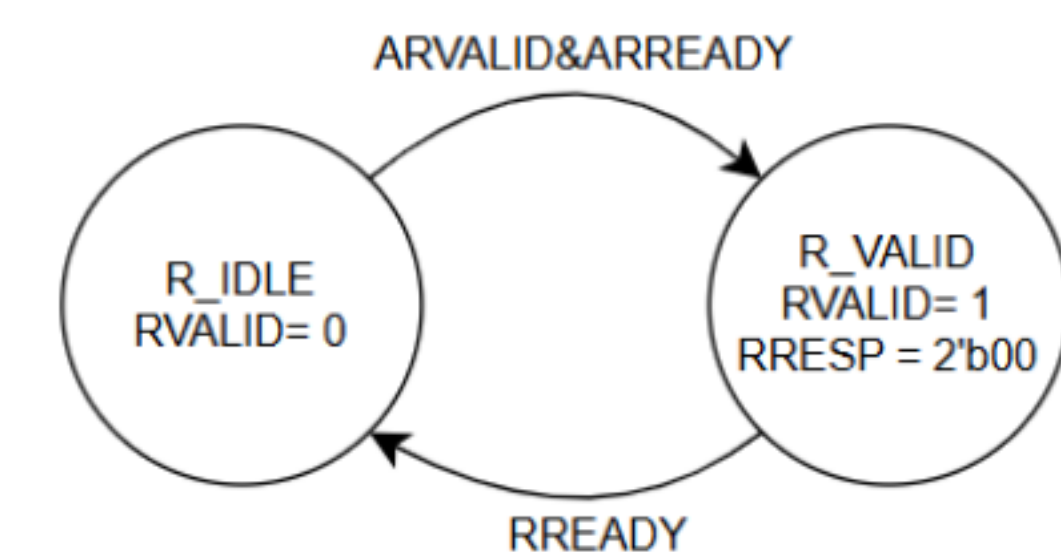
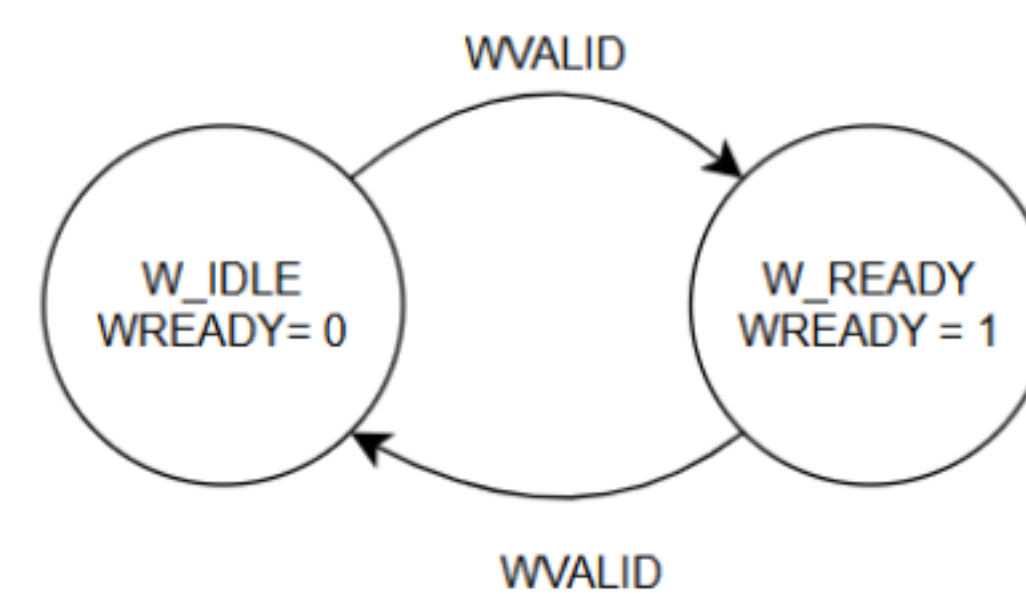
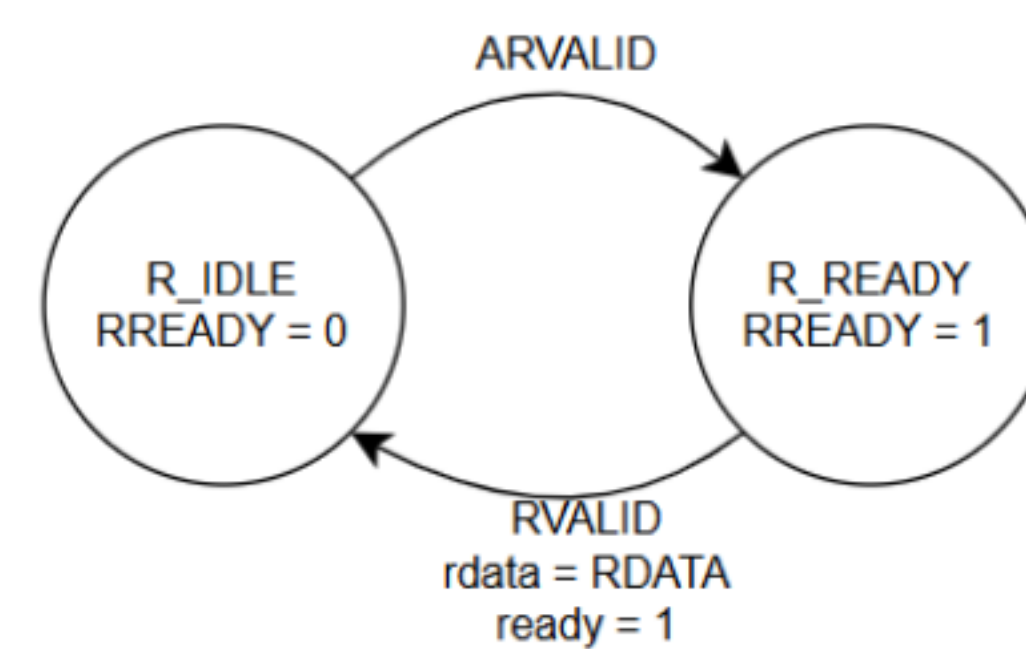
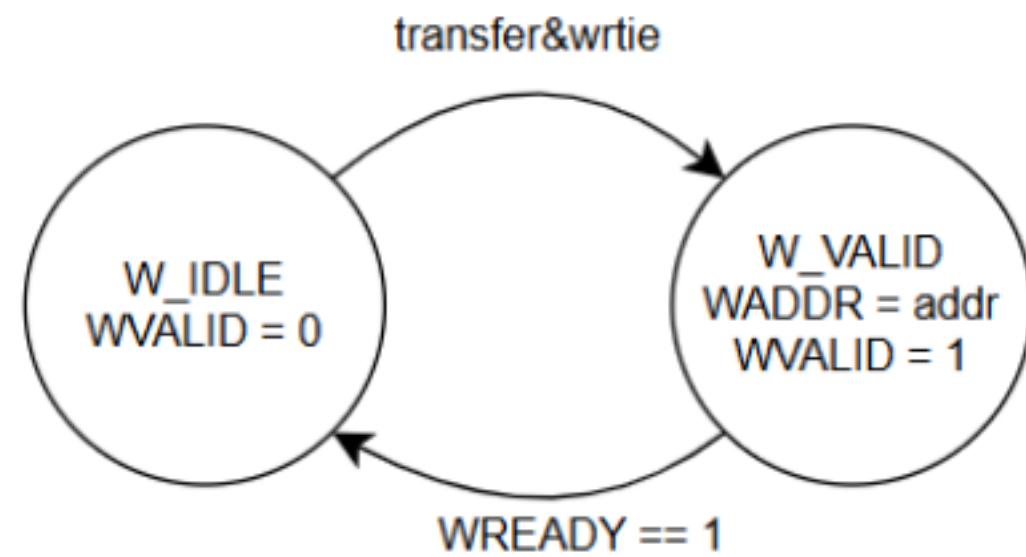
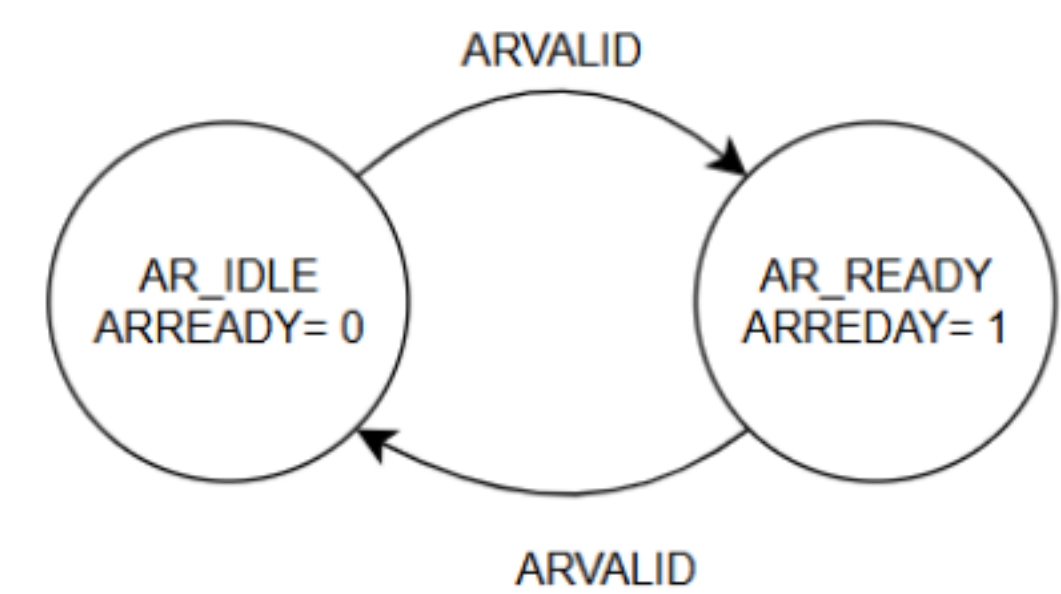
<Master> Read Transaction



<Slave> Write Transaction

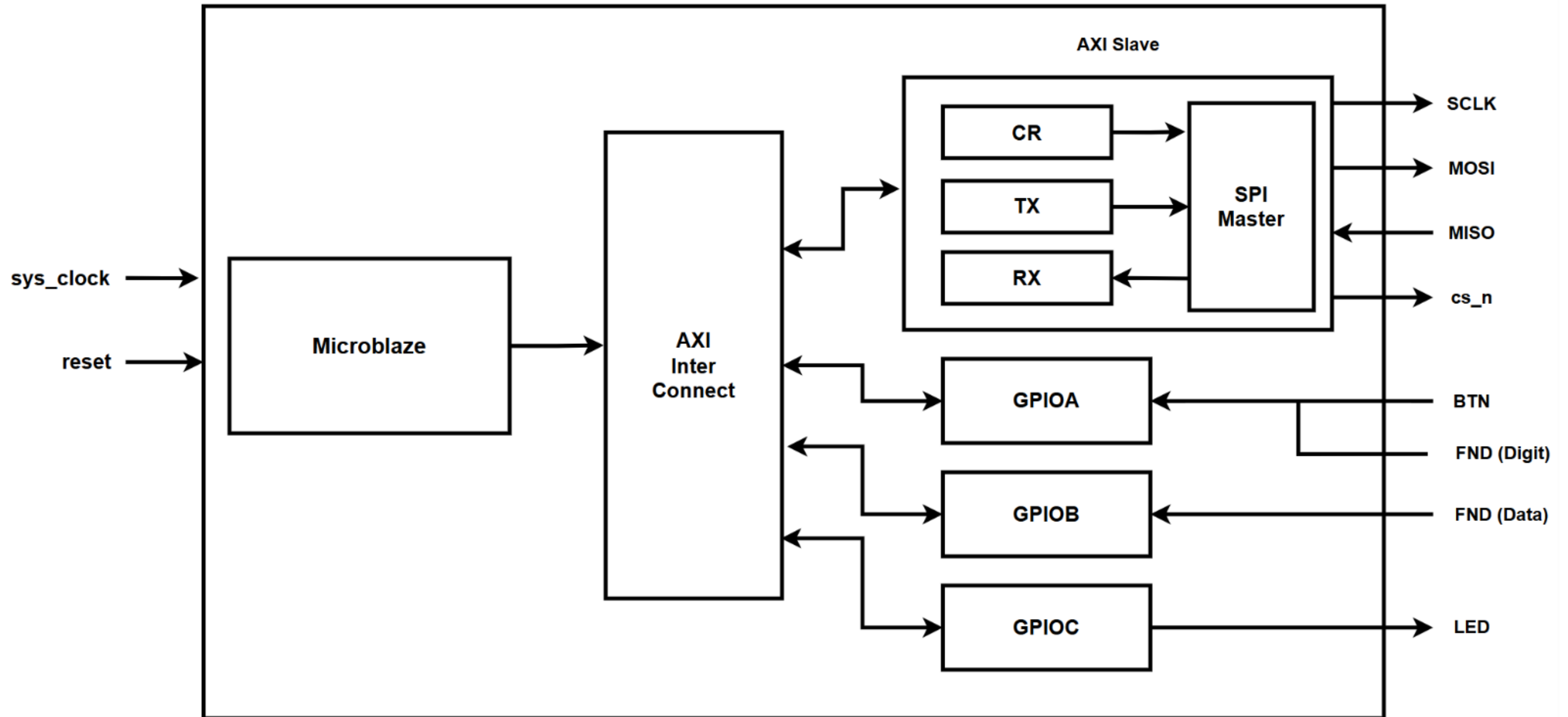


<Slave> Read Transaction

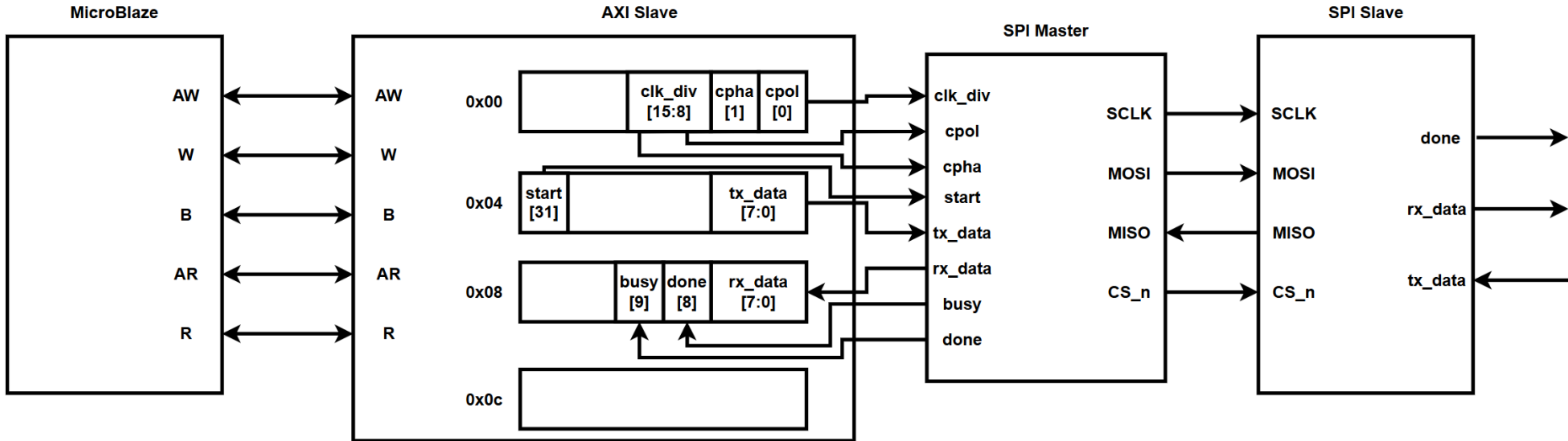


AXI+SPI

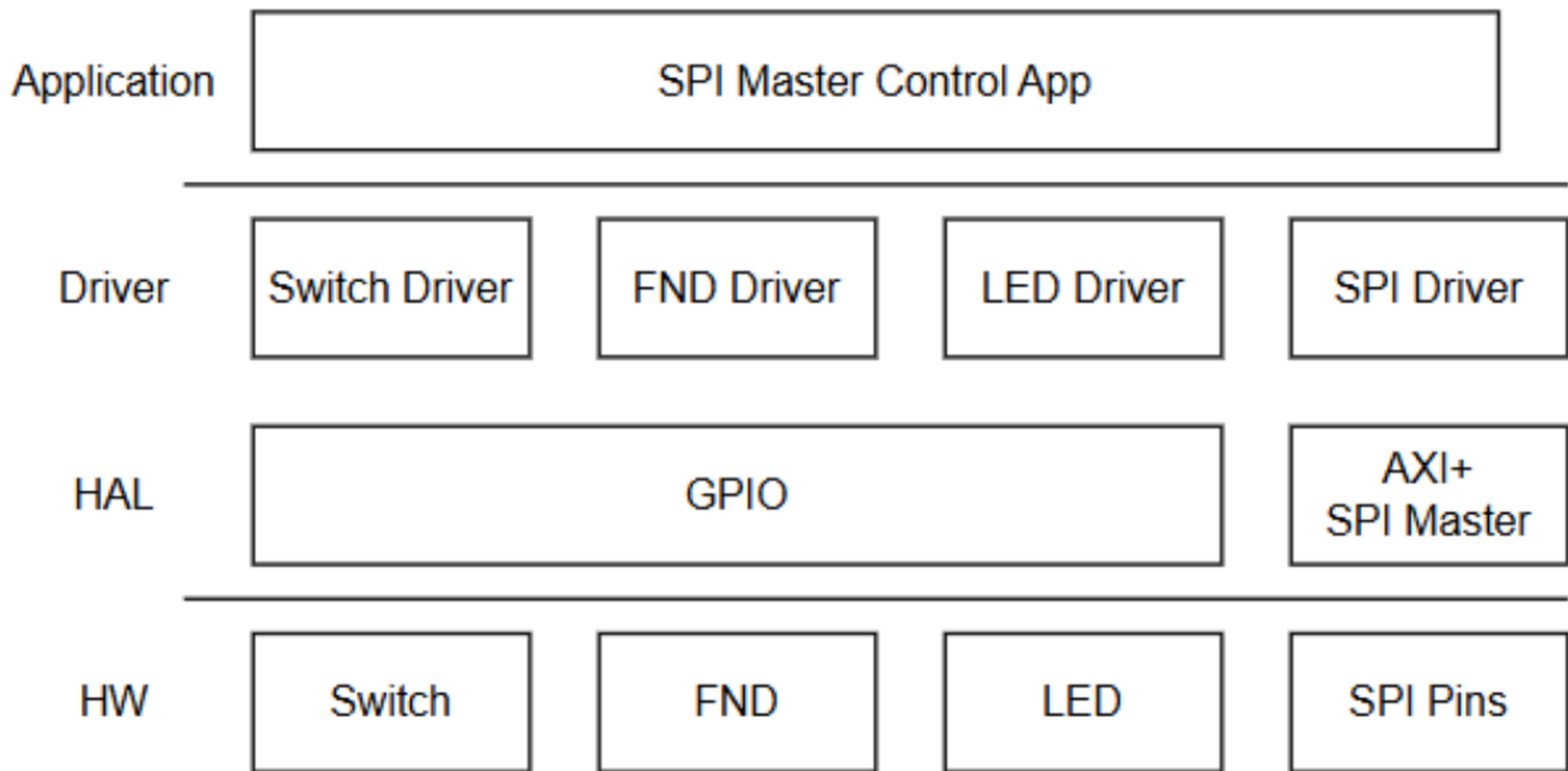
System Block Diagram



SPI/AXI Bridge Diagram

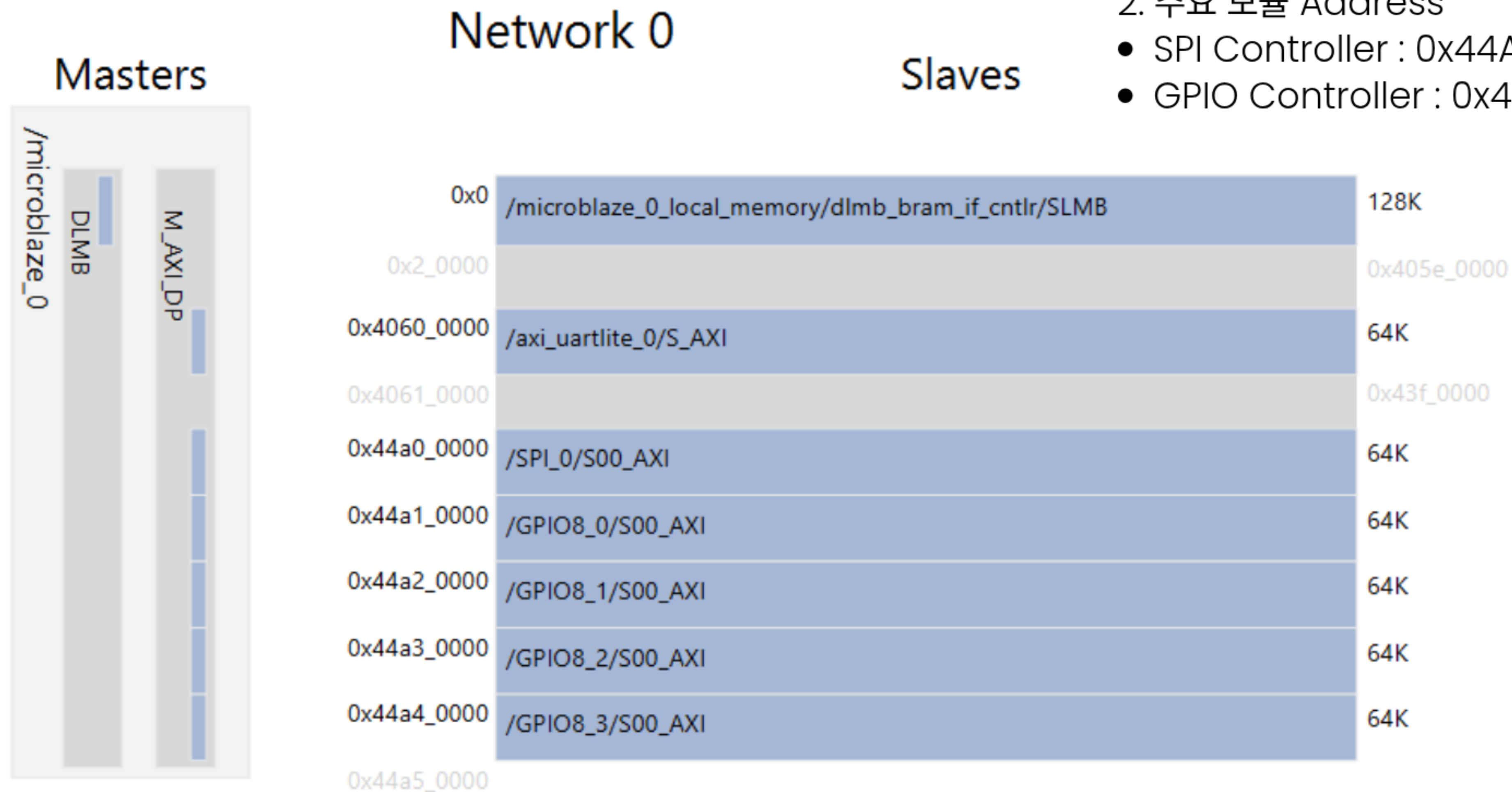


Layered_Architecture



1. Layered Architecture (계층형 구조)
상위 애플리케이션과 하위 하드웨어의 독립성 확보
2. Hardware Abstraction Layer (HAL)
복잡한 하드웨어 레지스터 접근을 추상화하여 개발 편의성 증대
3. Unified Driver Interface
SPI 및 GPIO(Switch, FND, LED)의 통합 관리 체계

Address Map



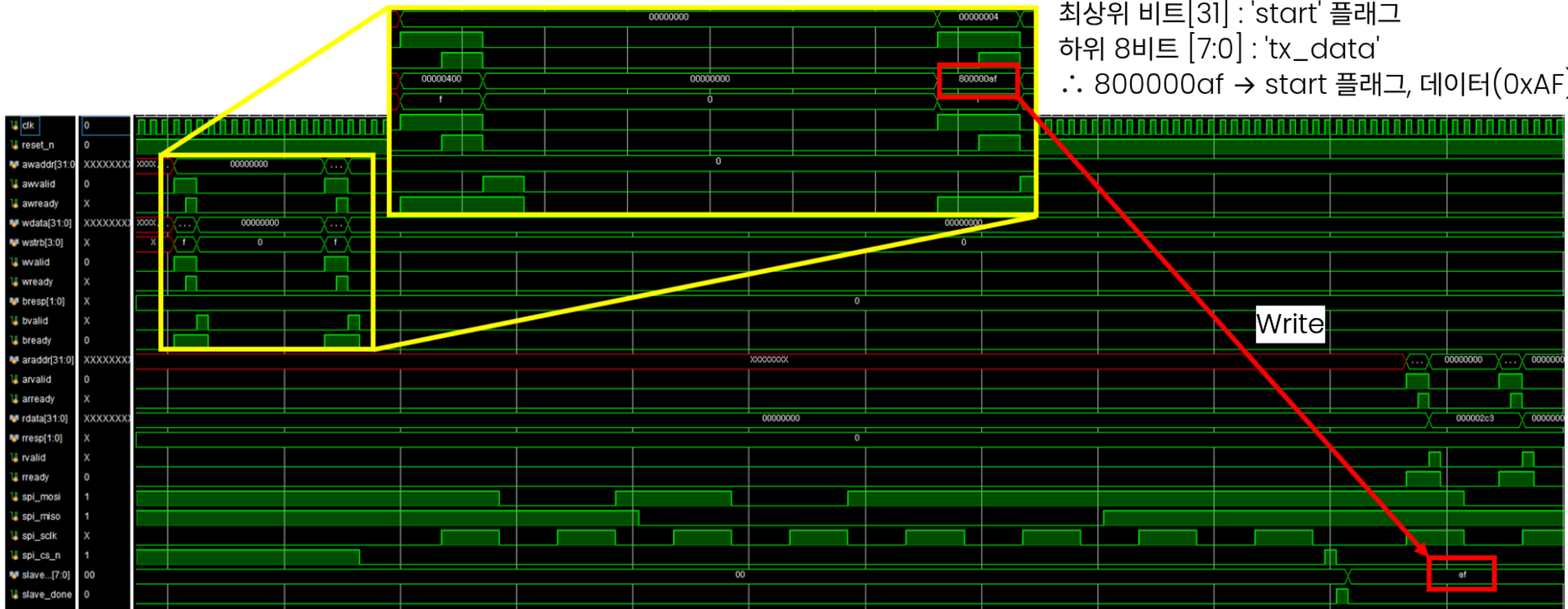
1. 고유 주소 할당

- AXI 디코더를 통해 모듈별 독립적인 메모리 공간 부여

2. 주요 모듈 Address

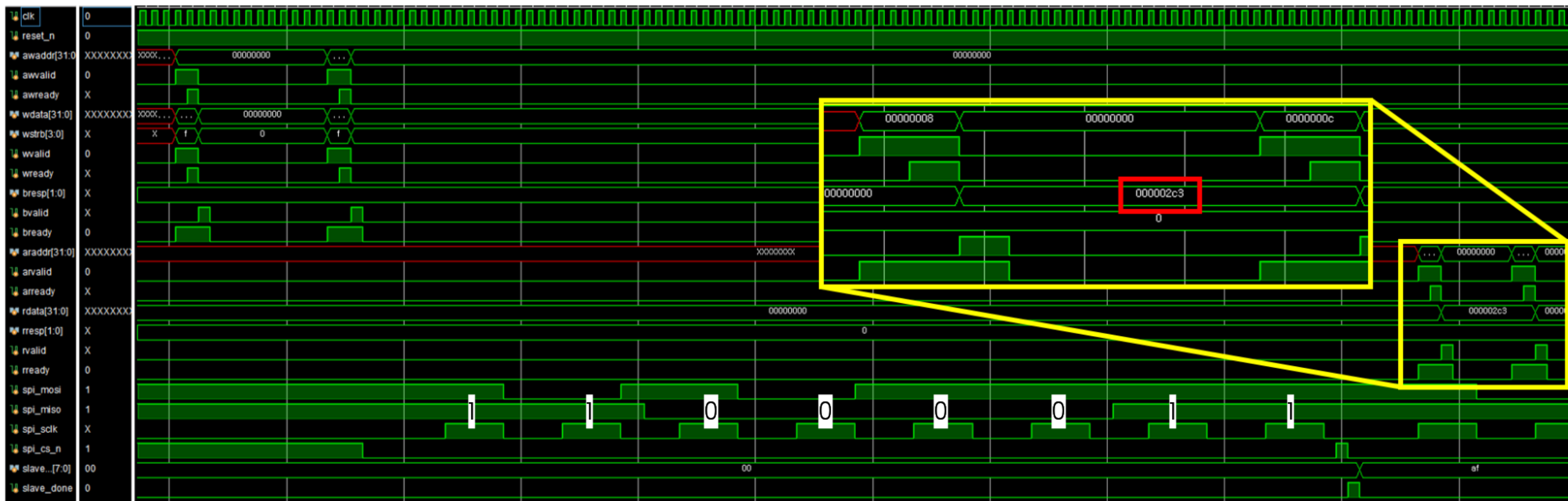
- SPI Controller : 0x44A0_0000
- GPIO Controller : 0x44A1_0000

Verification



Verification

하위 8비트(C3) = rx_data
상위 비트 = 상태 플래그 (8번 비트: Busy, 9번 비트: Done)



Busy = 1, Done = 0 -> (Dec) 1-0 : (Hex) 2
spi_miso(수신 핀) → (Dec) 1-1-0-0-0-0-1-1 : (Hex) C3

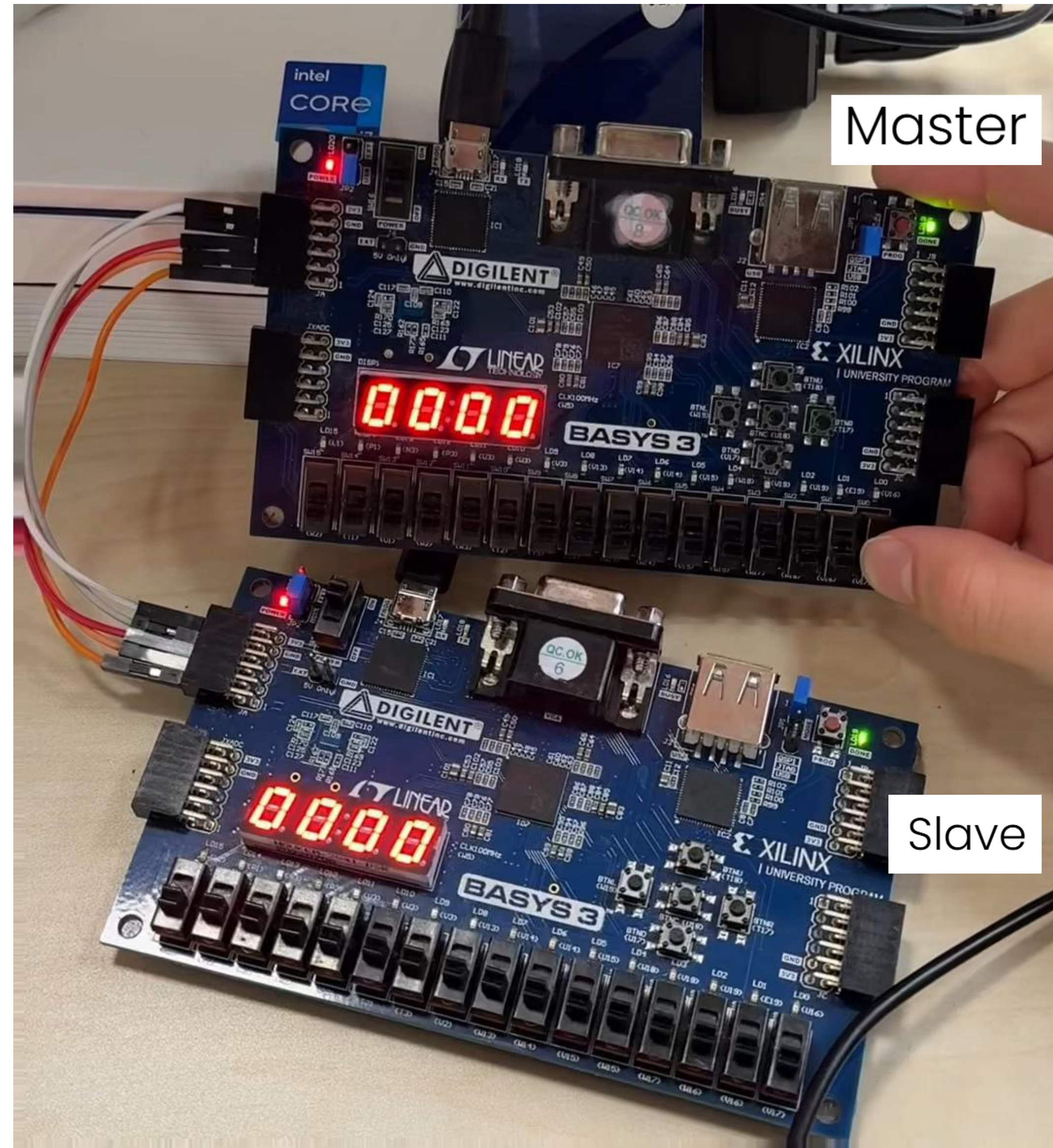
UVM Verification

```
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29794165000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [991/1000] Created -> Master TX: 0xeb
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29794205000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0xeb
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29810415000: uvm_test_top.env.mon [MON] [MON-CAPTURE #991] Slave Rcvd: 0xeb
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29810415000: uvm_test_top.env.scb [SCB_RESULT] [PASS #991] Sent: 0xeb == Rcvd: 0xeb
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29824255000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [992/1000] Created -> Master TX: 0x03
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29824295000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x03
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29840505000: uvm_test_top.env.mon [MON] [MON-CAPTURE #992] Slave Rcvd: 0x03
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29840505000: uvm_test_top.env.scb [SCB_RESULT] [PASS #992] Sent: 0x03 == Rcvd: 0x03
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29854345000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [993/1000] Created -> Master TX: 0x25
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29854385000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x25
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29870595000: uvm_test_top.env.mon [MON] [MON-CAPTURE #993] Slave Rcvd: 0x25
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29870595000: uvm_test_top.env.scb [SCB_RESULT] [PASS #993] Sent: 0x25 == Rcvd: 0x25
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29884435000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [994/1000] Created -> Master TX: 0xe4
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29884475000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0xe4
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29900685000: uvm_test_top.env.mon [MON] [MON-CAPTURE #994] Slave Rcvd: 0xe4
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29900685000: uvm_test_top.env.scb [SCB_RESULT] [PASS #994] Sent: 0xe4 == Rcvd: 0xe4
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29914525000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [995/1000] Created -> Master TX: 0x92
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29914565000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x92
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29930775000: uvm_test_top.env.mon [MON] [MON-CAPTURE #995] Slave Rcvd: 0x92
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29930775000: uvm_test_top.env.scb [SCB_RESULT] [PASS #995] Sent: 0x92 == Rcvd: 0x92
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29944615000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [996/1000] Created -> Master TX: 0xc1
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29944655000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0xc1
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29960865000: uvm_test_top.env.mon [MON] [MON-CAPTURE #996] Slave Rcvd: 0xc1
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29960865000: uvm_test_top.env.scb [SCB_RESULT] [PASS #996] Sent: 0xc1 == Rcvd: 0xc1
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 29974705000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [997/1000] Created -> Master TX: 0x35
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 29974745000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x35
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 29990955000: uvm_test_top.env.mon [MON] [MON-CAPTURE #997] Slave Rcvd: 0x35
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 29990955000: uvm_test_top.env.scb [SCB_RESULT] [PASS #997] Sent: 0x35 == Rcvd: 0x35
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 30004795000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [998/1000] Created -> Master TX: 0xf4
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 30004835000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0xf4
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 30004835000: uvm_test_top.env.mon [MON] [MON-CAPTURE #998] Slave Rcvd: 0xf4
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 30004835000: uvm_test_top.env.scb [SCB_RESULT] [PASS #998] Sent: 0xf4 == Rcvd: 0xf4
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 30004835000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [999/1000] Created -> Master TX: 0x64
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 30004835000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x64
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 30004835000: uvm_test_top.env.mon [MON] [MON-CAPTURE #999] Slave Rcvd: 0x64
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 30004835000: uvm_test_top.env.scb [SCB_RESULT] [PASS #999] Sent: 0x64 == Rcvd: 0x64
UVM_INFO ./tb/tb_axi_spi.sv(71) @ 30004835000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] Item [1000/1000] Created -> Master TX: 0x8b
UVM_INFO ./tb/tb_axi_spi.sv(131) @ 30004835000: uvm_test_top.env.drv [DRV] [DRV] Driving Item to AXI -> TX Data: 0x8b
UVM_INFO ./tb/tb_axi_spi.sv(171) @ 30004835000: uvm_test_top.env.mon [MON] [MON-CAPTURE #1000] Slave Rcvd: 0x8b
UVM_INFO ./tb/tb_axi_spi.sv(207) @ 30004835000: uvm_test_top.env.scb [SCB_RESULT] [PASS #1000] Sent: 0x8b == Rcvd: 0x8b
UVM_INFO ./tb/tb_axi_spi.sv(81) @ 30095065000: uvm_test_top.env.sqr@seq [SEQ] [SEQ] All AXI transactions generated successfully.
```

SPI AXI TEST SUMMARY REPORT

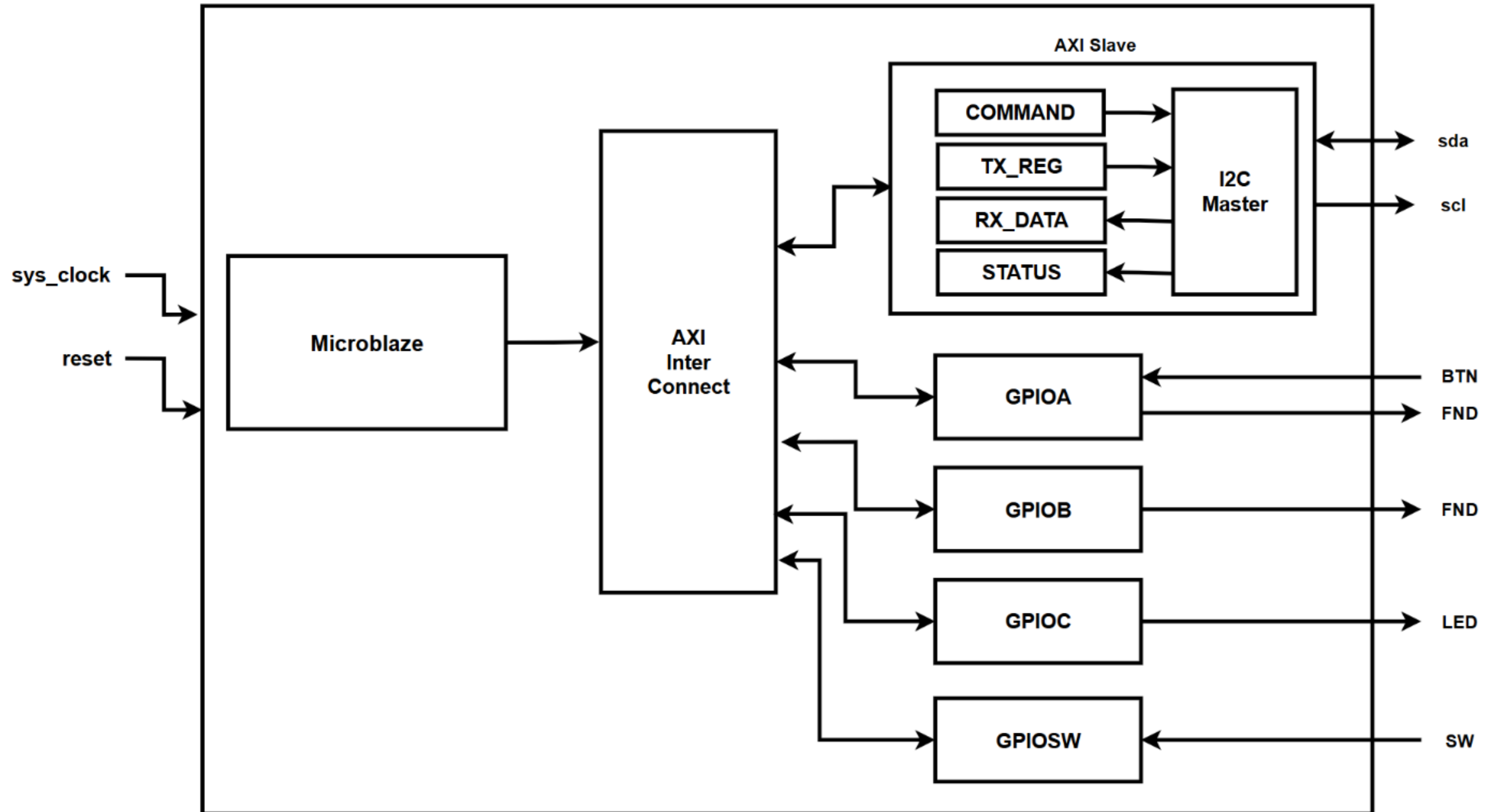
Total Transactions : 1000
Passed : 1000
Failed : 0

Demonstration

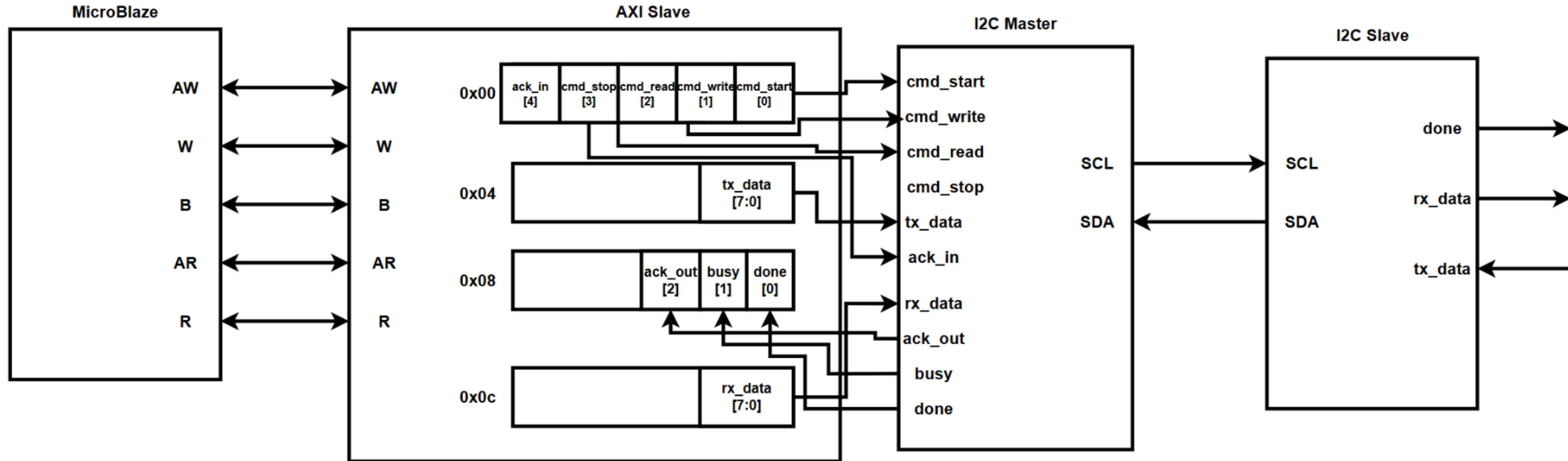


AXI+I2C

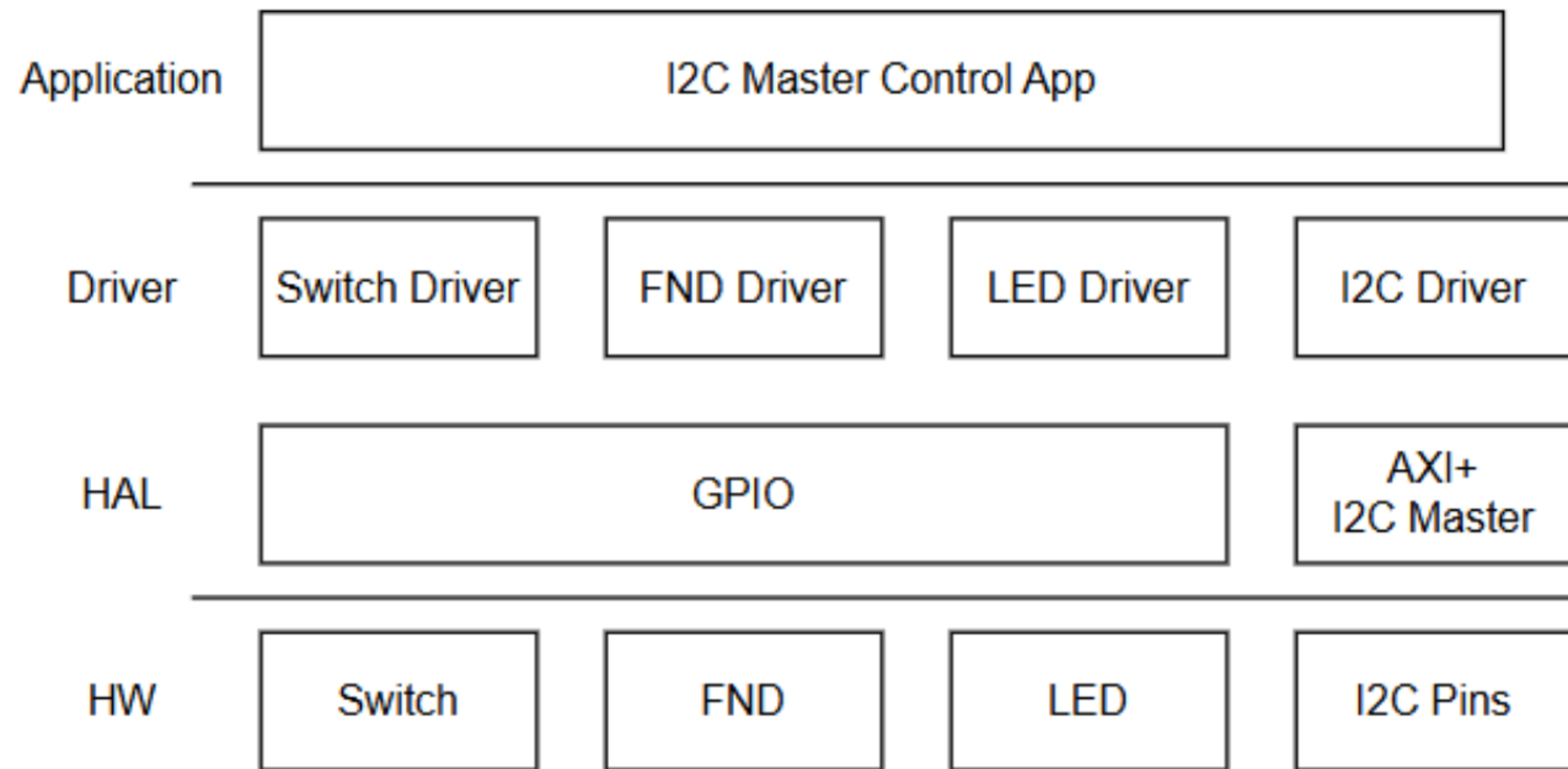
System Block Diagram



I2C/AXI Bridge Diagram

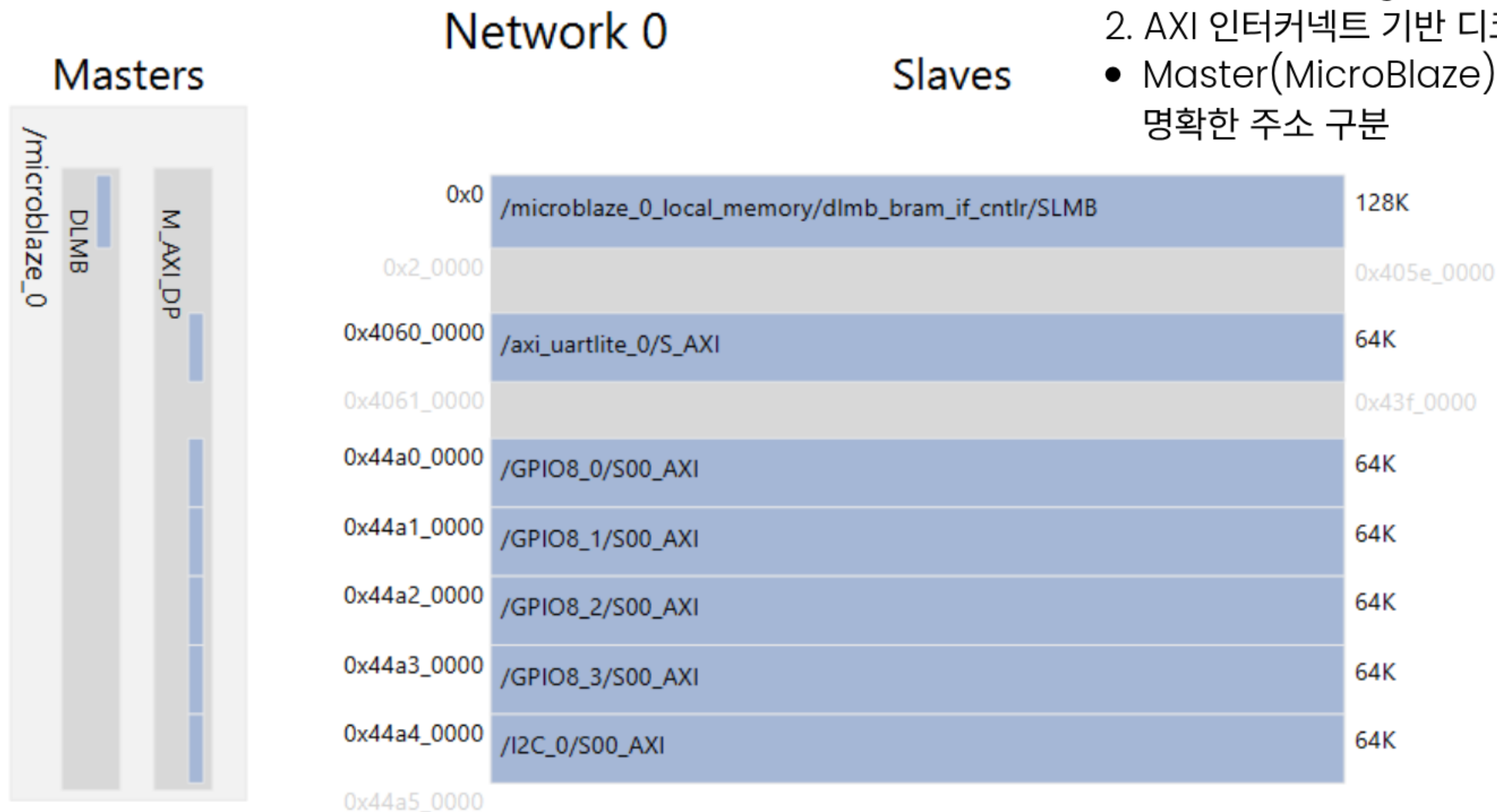


Layered_Architecture



1. Layered Architecture (계층형 구조)
 - 애플리케이션(App)과 하드웨어(HW)의 완벽한 분리
2. 하드웨어 추상화 (HAL&Driver)
 - AXI+I2C Master의 복잡한 레지스터 제어를 하위 계층으로 은닉
 - 상위 앱에서는 단순화된 API만 호출하여 통신 수행
3. 통합 제어 환경 구축
 - I2C 통신 및 I/O 장치(Switch, FND, LED)를 동일한 체계로 관리

Address Map



1. 독립적 주소 공간 할당

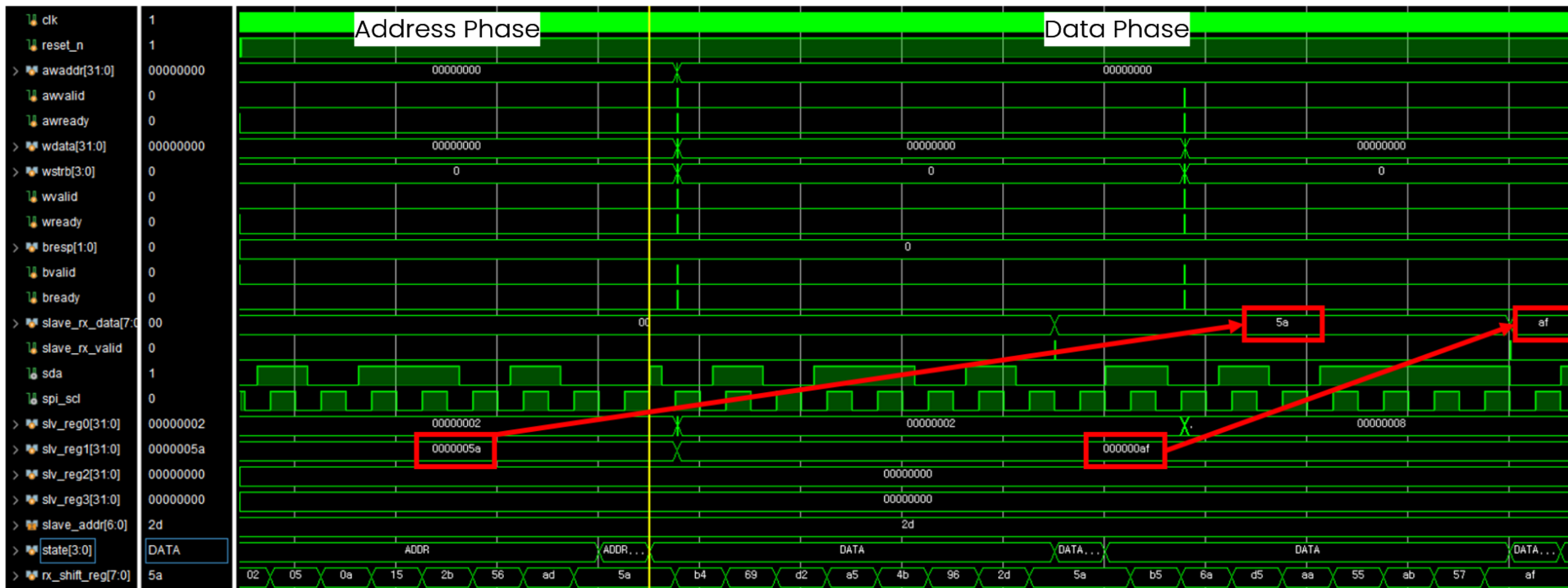
- I2C Master 전용 Base Address : 0x44A5_0000

- Address Range : 64KB 공간 확보

2. AXI 인터커넥트 기반 디코딩

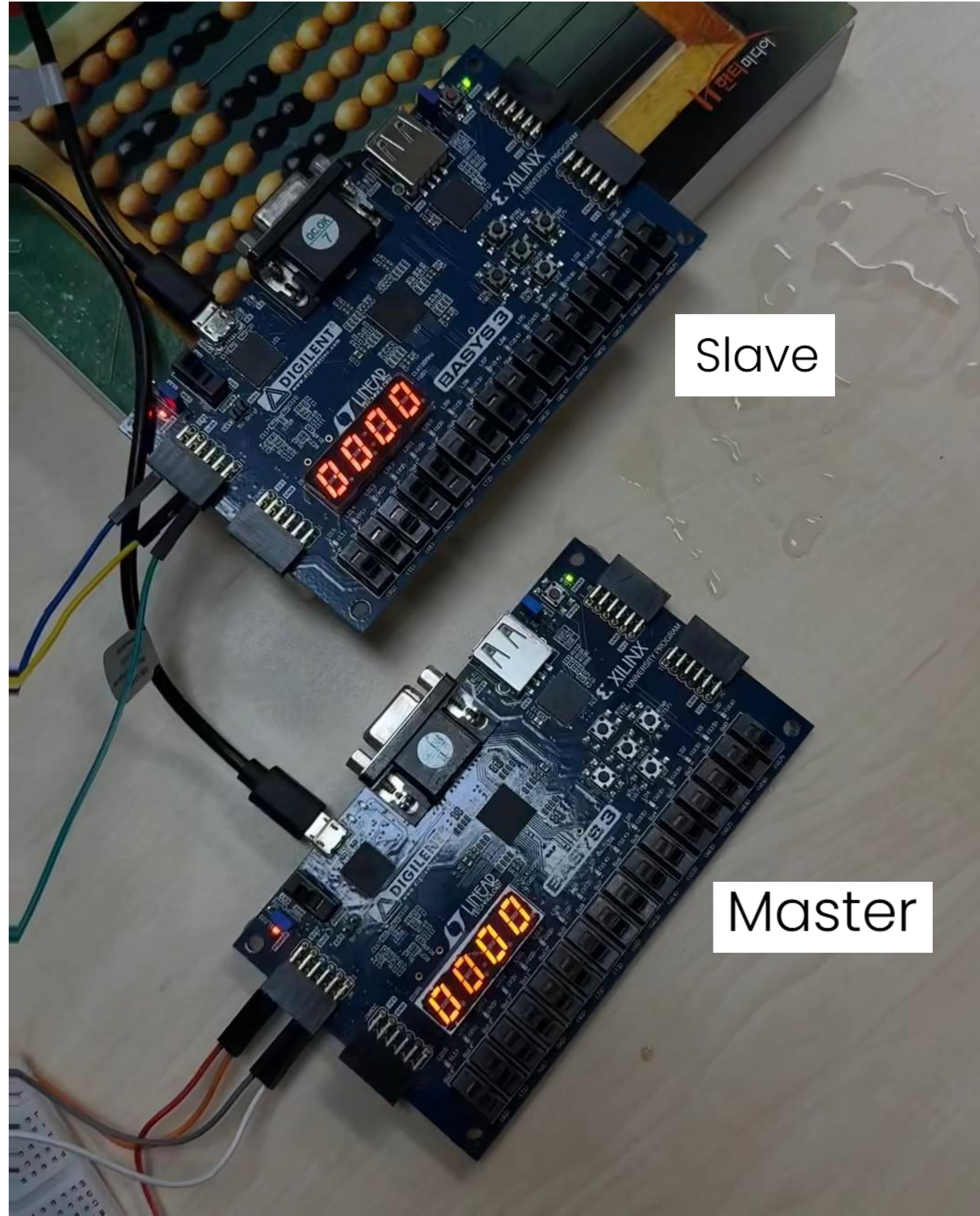
- Master(MicroBlaze)와 Slave(I2C,UART,GPIO) 간의 명확한 주소 구분

Verification



Master → Slave의 데이터 흐름 (Data Flow)

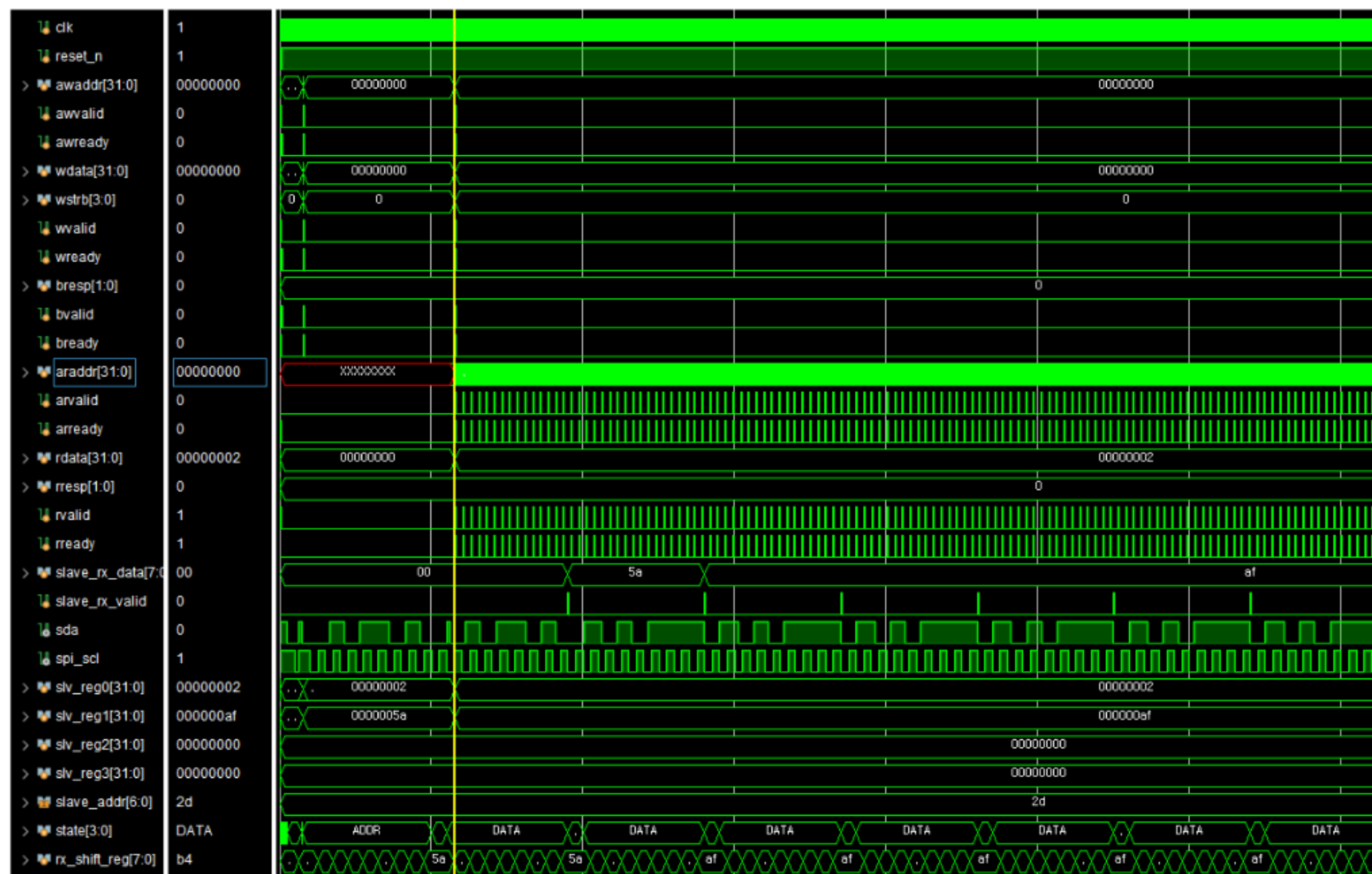
Demonstration



Trouble Shooting

01. 문제 상황

- I2C 데이터 수신 시, 값이 누락되거나 이전 통신의 쓰레기값이 읽히는 현상 발생
- 시뮬레이션 파형 분석 결과, 수신 데이터가 완성되기 전 상태 레지스터에 접근하는 오류 포착 (Busy=1, Done=0)



Trouble Shooting

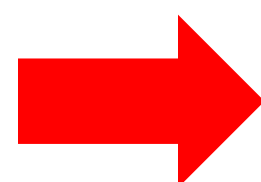
02. 원인 분석

- 하드웨어-소프트웨어 처리 속도 비동기화
- 수백 MHz로 동작하는 프로세서(AXI 버스)가 수십~수백 kHz로 동작하는 물리적 I2C 통신의 완료 시점을 기다리지 않고 즉각적인 Read 명령을 수행하여 타이밍 불일치 발생

03. 해결 방안

- 상태 레지스터 기반의 Polling 동기화 로직 구현
- 소프트웨어 단에서 0x08번지의 Done 비트를 지속적으로 모니터링
- 하드웨어 트랜잭션이 완벽히 종료된 것을 확인한 후 수신 데이터를 읽어오도록 코드 수정

```
//I2C_WriteReg(I2C_REG_CMD, I2C_CMD_READ | I2C_CMD_ACK_IN);  
//delay_us(2000);  
//rx_data = I2C_ReadReg(I2C_REG_RX_DATA);  
//I2C_WriteReg(I2C_REG_CMD, 0x00);
```



```
I2C_WriteReg(I2C_REG_CMD, I2C_CMD_READ | I2C_CMD_ACK_IN);  
  
uint32_t status;  
do {  
    status = I2C_ReadReg(I2C_REG_STATUS);  
} while ((status & DONE_BIT_MASK) == 0);  
  
rx_data = I2C_ReadReg(I2C_REG_RX_DATA);  
I2C_WriteReg(I2C_REG_CMD, 0x00);
```

Reflections

- 처음에 프로젝트를 준비하기 시작하면서 하드웨어와 소프트웨어 사이의 미세한 타이밍을 맞추는 게 막막하고 힘들었지만, 파형을 분석하며 문제를 해결할 수 있어서 뿌듯했다.
 - 설계도대로 완벽할 것 같던 하드웨어가 실제 보드에서 오작동하는 것을 보며, 실무 설계는 아주 작은 변수까지 계산해야 한다는 점을 실감했다.
-

